~\Desktop\C++学习资料\Untitled-1.cpp

```cpp
#include <cuda_runtime.h>
#include <iostream>
// 核心 Kernel：假设 blockDim.x 是 2 的幂次 (例如 128, 256, 512)
__global__ void reduceInterleaved(const float *g_idata, float *g_odata, int n) {
    // 1. 动态分配共享内存，大小由 Host 端启动时决定
    extern __shared__ float sdata[];
    // 计算全局索引和块内索引
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
    // 2. 加载数据到共享内存 (包含边界检查)
    // 只有第一次从 Global Mem 读取时需要检查 n，后续都在 Shared Mem 中操作
    sdata[tid] = (i < n) ? g_idata[i] : 0.0f;
    __syncthreads();
    // 3. 核心循环：连续地址归约 (Sequential Addressing)
    // 步长 s 从 blockDim/2 开始，每次减半
    // 关键点：这里没有使用取模 (%) 操作，效率极高
    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            // 只有前 s 个线程是活跃的，且访问的是 sdata[tid] 和 sdata[tid+s]
            // 这种模式极大减少了 Shared Memory 的 Bank Conflict
            sdata[tid] += sdata[tid + s];
        }
        // 每一轮必须同步，确保上一轮计算完成
        __syncthreads();
    }
    // 4. 将本 Block 的归约结果写回 Global Memory
    if (tid == 0) g_odata[blockIdx.x] = sdata[0];
}
int main() {
    int N = 1 << 24; // 16M 元素
    int threadsPerBlock = 256;
    // 计算需要的 Block 数量
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

    size_t size = N * sizeof(float);
    float *d_in, *d_out;
    // 省略 Host 端初始化...
    cudaMalloc(&d_in, size);
    cudaMalloc(&d_out, blocksPerGrid * sizeof(float));
    // ★★★ 启动 Kernel ★★★
    // 第三个参数 threadsPerBlock * sizeof(float) 是动态分配共享内存的大小
    reduceInterleaved<<<blocksPerGrid, threadsPerBlock, threadsPerBlock * sizeof(float)>>>(d_in, d_out, N);
    // 注意：d_out 现在包含 blocksPerGrid 个部分和，通常需要在 Host 端再做一次求和
    // 或者递归调用 Kernel 直到结果为一个值
    cudaFree(d_in); cudaFree(d_out);
    std::cout << "Reduction Kernel Executed!" << std::endl;
    return 0;
}
```